

SphereLab.ai

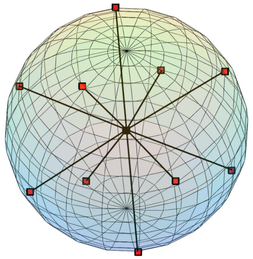
Scalable Principles
for Learning and Reasoning Lab

Efficient Training of Large Language Models via Orthogonal Transformation

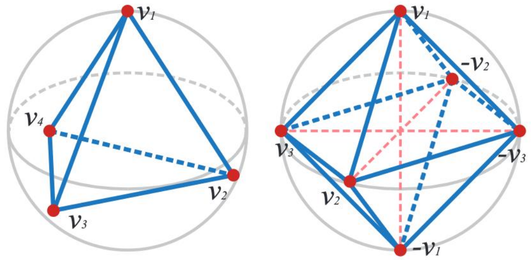
Weiyang Liu

Department of Computer Science and Engineering
The Chinese University of Hong Kong

Overview of Orthogonal Training



Hyperspherical Uniformity
(MHE, NeurIPS 2018)
(HyperUni, AISTATS 2021)

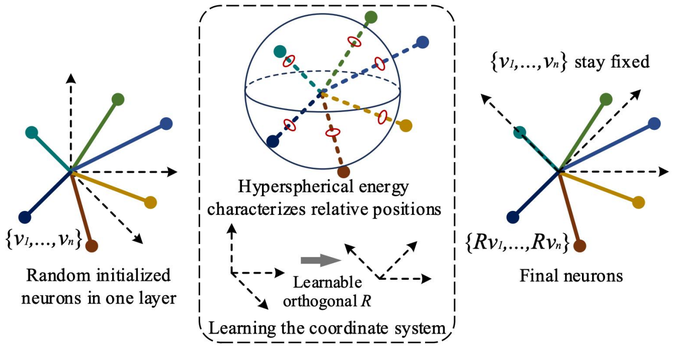


Hyperspherical Uniformity Gap
(GNC, ICLR 2023)

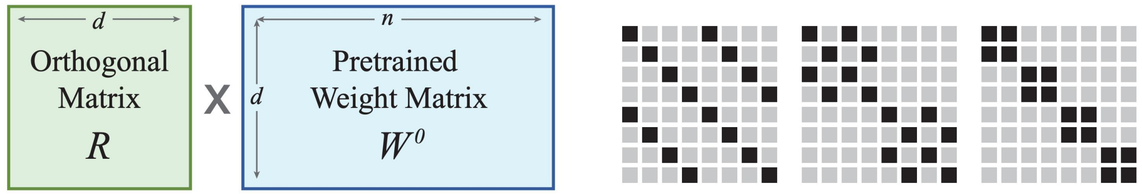
Pretraining

Guiding first principle

Finetuning



Orthogonal Pretraining
(OPT, CVPR 2021) (POET, NeurIPS 2025)
(POET-X, ICML 2026) (Pion, NeurIPS 2026)



Orthogonal Finetuning
(OFT, NeurIPS 2023)
(OFTv2, EMNLP 2025)

Orthogonal Butterfly
(BOFT, ICLR 2024)

Parameter-efficient generalization

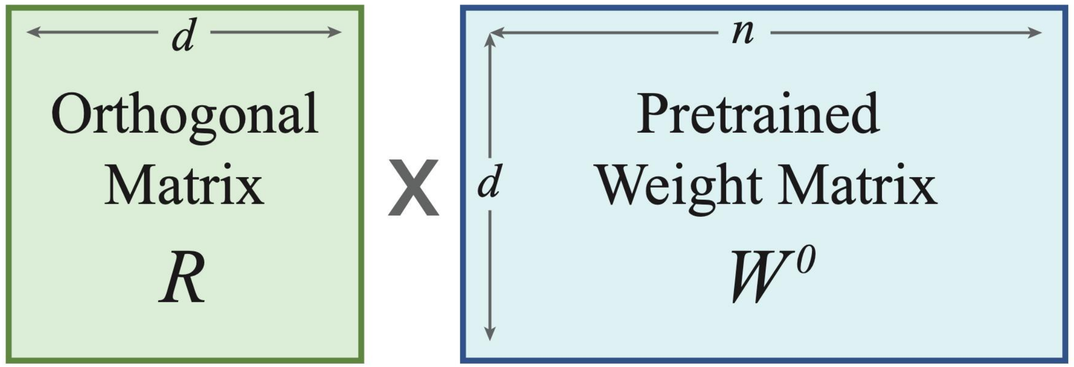
Orthogonal Finetuning

- **Key idea**: Angular information in neurons preserves semantics, so finetuning should preserve the angles between neurons.
- **Method**: Learn orthogonal multiplicative weight update
- **Hyperspherical energy** does not change under the orthogonal transformation.

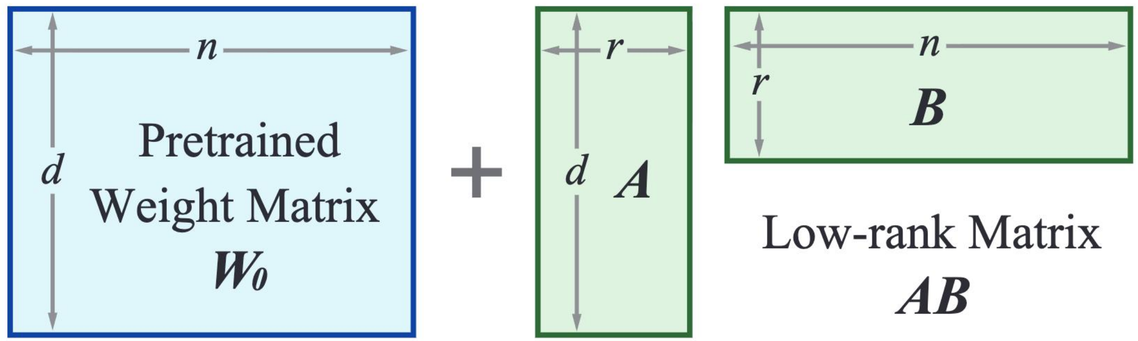
$$\sum_{i \neq j} \left\| \frac{w_i}{\| w_i \|} - \frac{w_j}{\| w_j \|} \right\|^2$$

Normalized neuron weights

Sum over neurons in the same layer

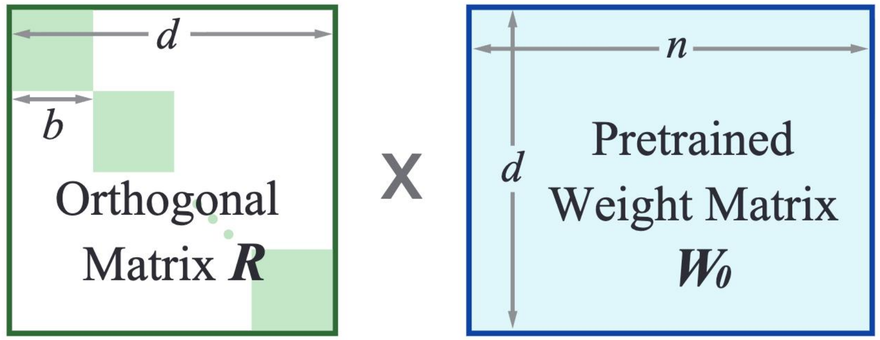


How OFT Saves Parameters



(a) Low-rank Structure in LoRA

Low-rankness



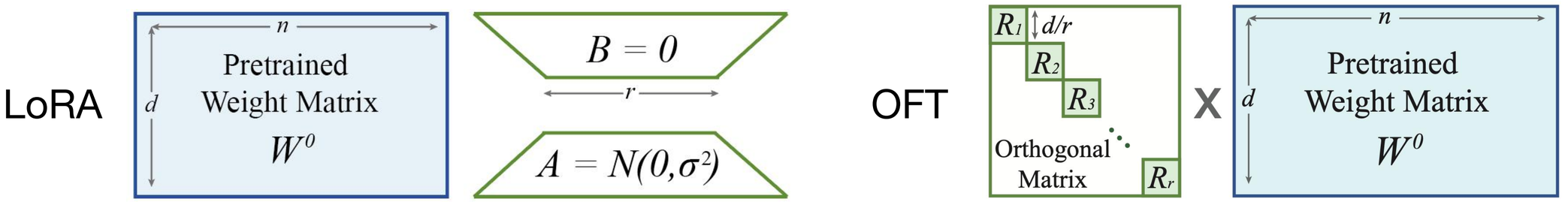
(b) Sparse Orthogonal Structure in OFT

Block-diagonal sparsity

Controlling Text-to-Image Diffusion by Orthogonal Finetuning, NeurIPS 2023

Orthogonal Finetuning (OFT) v.s. Low-Rank Adaptation (LoRA)

- LoRA uses a low-rank additive weight update, while OFT uses an orthogonal multiplicative update:



- Parameter efficiency
  - LoRA uses **low-rankness**
  - OFT uses **sparsity**

- Duality between LoRA and OFT:

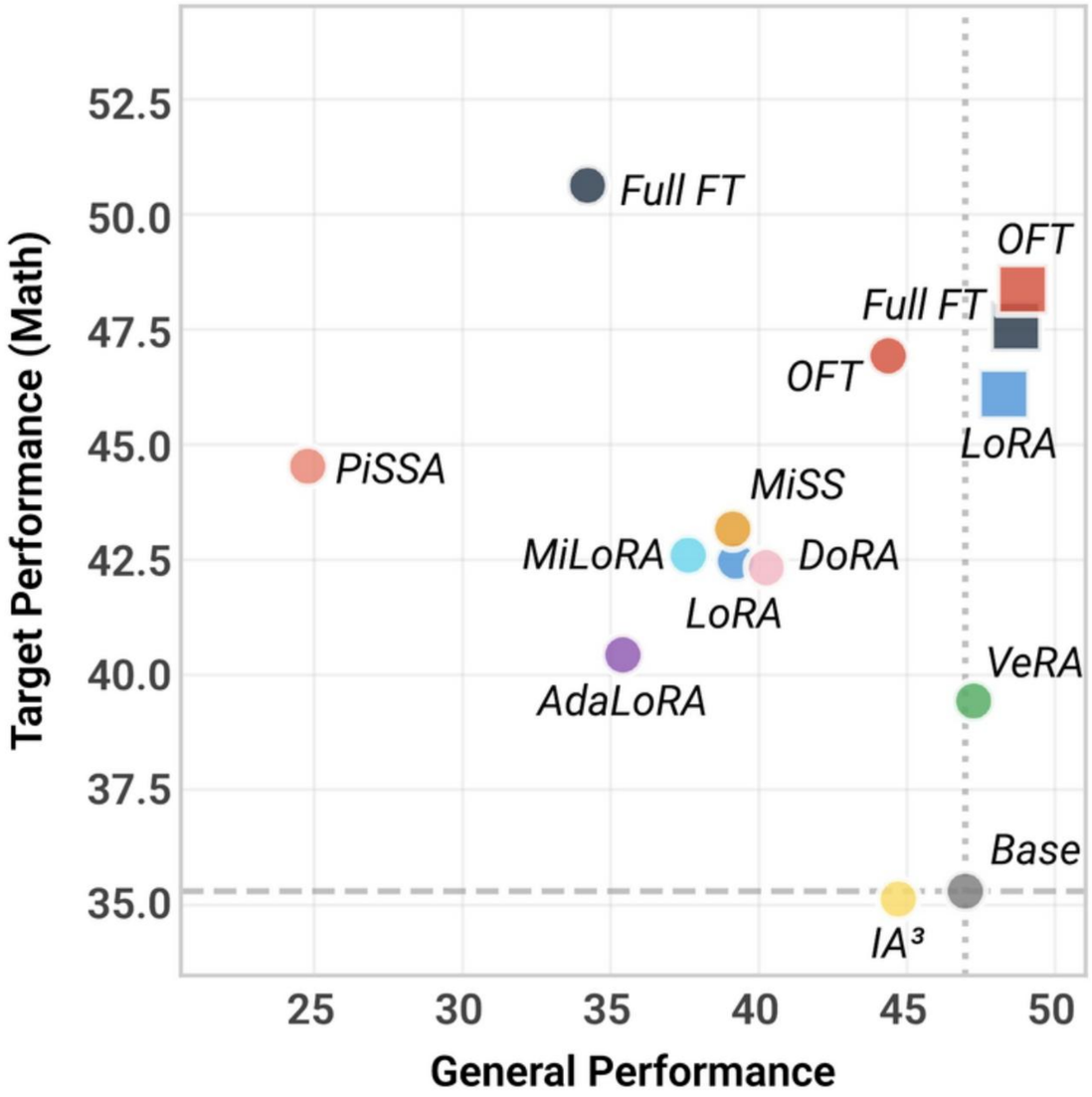
$$W_{new} = W + B \cdot A$$ Minimal information added

$$W_{new} = R \cdot W$$ No information loss

LoRA: Low-Rank Adaptation of Large Language Models, ICLR 2022

Controlling Text-to-Image Diffusion by Orthogonal Finetuning, NeurIPS 2023

OFT excels at avoiding catastrophic forgetting



SFT post-training performance:

OFT achieves the best trade-off

RL post-training performance rank:

OFT > Full Finetuning > LoRA

OFTv2: Scalable Matrix-free Implementation

(1) **Weight transform**: matrix-matrix mult.

$$\boldsymbol{z} = \underbrace{\overbrace{\boldsymbol{W}_0^\top \boldsymbol{R}^\top}\, \boldsymbol{x}}_{\text{(2) Linear map: matrix-vector mult.}}$$

$O(n^3)$



(1) **Linear map**: matrix-vector mult.

$$\boldsymbol{z} = \underbrace{\boldsymbol{W}_0^\top \overbrace{\boldsymbol{R}^\top \boldsymbol{x}}}_{\text{(2) Linear map: matrix-vector mult.}}$$

$O(n^2)$

OFTv2: Cayley-Neumann Parameterization

Skew-symmetric matrix

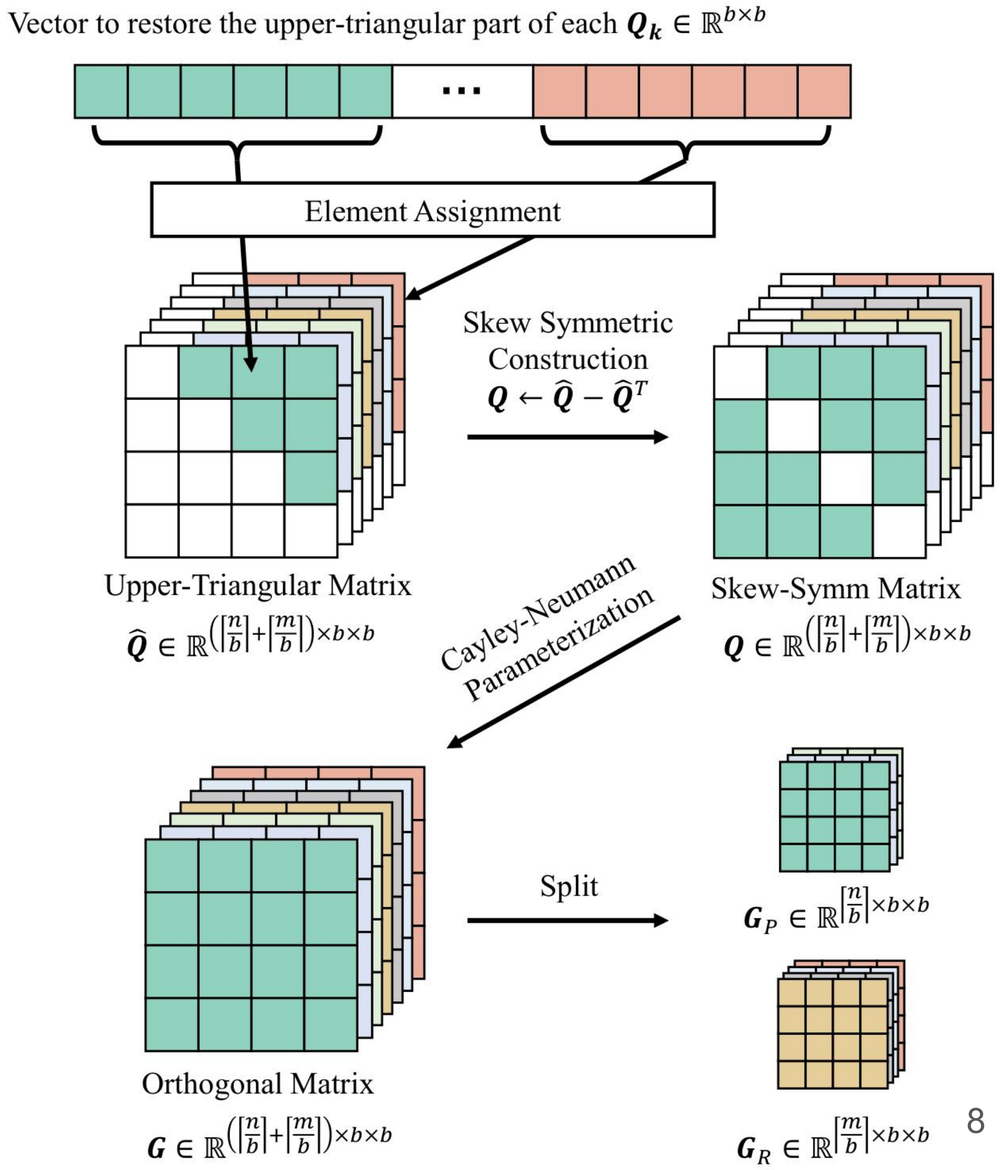
$$R = (I + Q)(I - Q)^{-1} = (I + Q)\left(\sum_{i=0}^{\infty} Q^i\right)$$

Cayley parameterization

Neumann series

$$\approx (I + Q)\left(\sum_{i=0}^{k} Q^i\right)$$

Approximation



OFTv2: Cayley-Neumann Parameterization

$$(I + Q)(\sum_{i=0}^{3} Q^i) = 2(Q + Q^2 + Q^2 \cdot Q) + Q^2 \cdot Q^2 + I$$

$$\mathbf{G} \approx 2(\mathbf{Q} + \mathbf{Q}^2 + \mathbf{Q}^2 \cdot \mathbf{Q}) + \mathbf{Q}^2 \cdot \mathbf{Q}^2 + \mathbf{I}$$

Forward computation

$$\nabla_1 = \frac{\partial f}{\partial \mathbf{G}}, \quad \nabla_2 = \nabla_1 \mathbf{Q}^\top + \mathbf{Q}^\top \nabla_1$$

$$\nabla_3 = \nabla_1 (\mathbf{Q}^2)^\top + \mathbf{Q}^\top \nabla_2, \quad \nabla_4 = \nabla_2 (\mathbf{Q}^2)^\top + (\mathbf{Q}^2)^\top \nabla_2$$

$$\begin{aligned} \frac{\partial f}{\partial \mathbf{Q}} &= 2\nabla_1 + 2\nabla_2 + 2\nabla_3 + \nabla_4 \\ &= 2(\nabla_1 + \nabla_2) + (2\mathbf{Q}^\top + (\mathbf{Q}^2)^\top)\nabla_2 + (2\nabla_1 + \nabla_2)(\mathbf{Q}^2)^\top \end{aligned}$$

Backward computation

Only need to compose Q and Q^2

QOFT: Finetuning Quantized Base Models

QLoRA

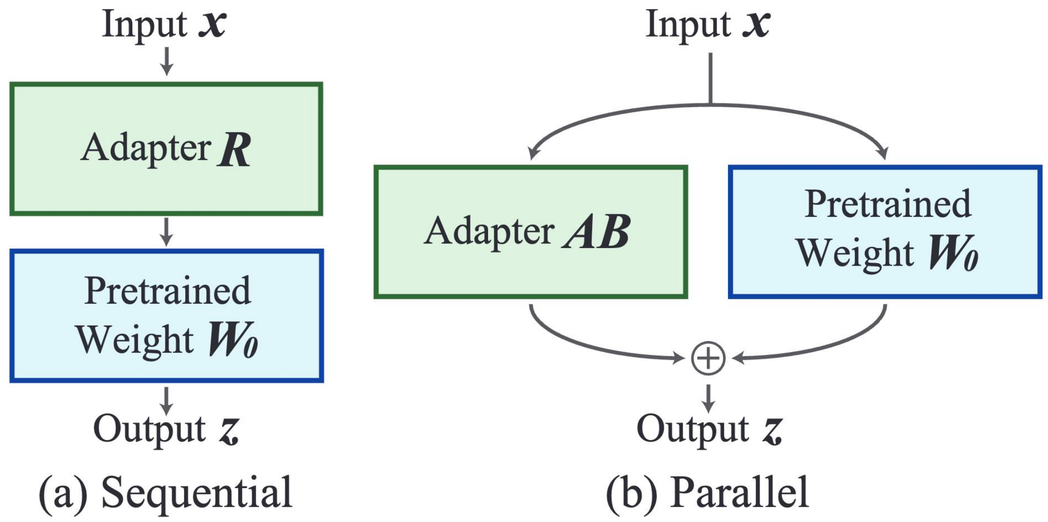
$$\boldsymbol{z} = \text{Dequant}(\boldsymbol{W}_{\text{quant}})^\top \boldsymbol{x} + (\boldsymbol{AB})^\top \boldsymbol{x}$$

Frozen Trainable

QOFT

$$\boldsymbol{z} = \text{Dequant}(\boldsymbol{W}_{\text{quant}})^\top \boldsymbol{R}^\top \boldsymbol{x}$$

Frozen Trainable

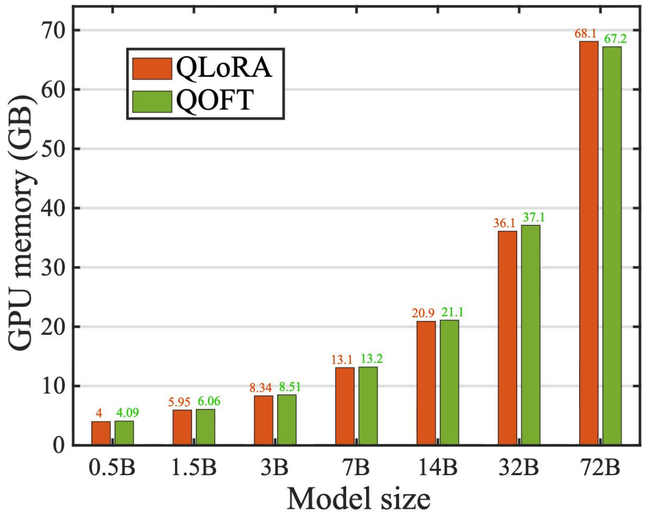


The worse-case requantization error for QLoRA is always larger than QOFT by $\|AB\|_\infty$

QOFT vs. QLoRA

Model	Type	# Params	AMC23	AQUA	CMATH	GaoKao 2023 En	Minerva Math	Olympiad/ Bench	SAT Math
Qwen2.5-1.5B-it	**Baseline**	-	17.5	49.2	65.2	36.4	9.6	12.0	59.4
	QLoRA	18.46M	15.0	42.5	61.5	29.6	8.1	8.9	59.4
	QOFT	7.89M	**27.5**	**53.1**	**68.5**	**41.0**	**11.8**	**14.4**	**81.2**
Qwen2.5-1.5B	**Baseline**	-	0.0	18.9	4.0	4.2	2.6	2.4	28.1
	QLoRA	18.46M	15.0	37.4	**64.2**	26.8	**8.5**	6.8	62.5
	QOFT	7.89M	**22.5**	**53.1**	56.3	**36.1**	**8.5**	**12.7**	**87.5**
Qwen2.5-7B-it	**Baseline**	-	50.0	16.5	89.3	61.8	**33.5**	36.6	53.1
	QLoRA	40.37M	30.0	48.0	88.8	50.1	25.4	19.7	68.8
	QOFT	17.55M	**52.5**	**70.9**	**90.5**	**63.6**	**33.5**	**37.6**	**96.9**
Qwen2.5-7B	**Baseline**	-	25.0	55.1	61.2	42.9	11.8	29.9	71.9
	QLoRA	40.37M	35.0	48.8	73.7	49.9	18.8	18.5	62.5
	QOFT	17.55M	**52.5**	**59.4**	**80.7**	**55.6**	**21.7**	**34.7**	**87.5**
Qwen2.5-32B-it	**Baseline**	-	62.5	18.5	92.5	70.1	**41.5**	44.4	65.6
	QLoRA	134.22M	62.5	71.7	94.0	71.2	39.7	46.8	96.9
	QOFT	57.90M	**75.0**	**83.1**	**94.7**	**73.5**	**41.5**	**48.7**	**100.0**
Qwen2.5-32B	**Baseline**	-	35.0	23.2	35.7	46.8	20.2	25.2	62.5
	QLoRA	134.22M	40.0	52.4	90.5	61.0	32.0	29.8	65.6
	QOFT	57.90M	**70.0**	**68.5**	**90.7**	**71.4**	**36.0**	**44.9**	**93.8**

Finetuning performance



GPU memory

Model Size	GPUs	QLoRA	QOFT
Qwen2.5-1.5B	8×H100	01:20:00	**01:17:30**
Qwen2.5-7B	8×H100	03:25:00	**03:19:30**
Qwen2.5-32B	8×H100	12:51:45	**12:27:45**

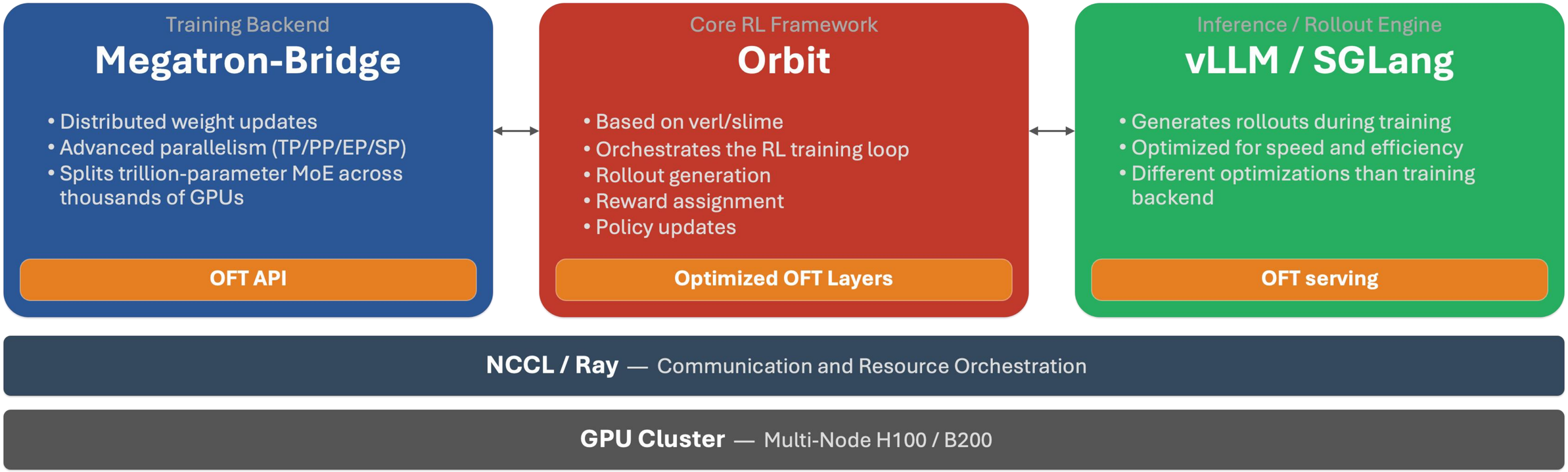
Training time



Memory-efficient RL Infrastructure with Orthogonal Finetuning

spherelab.ai/orbit

Orbit: OFT-centric Post-training Infra for Trillion-scale LLMs



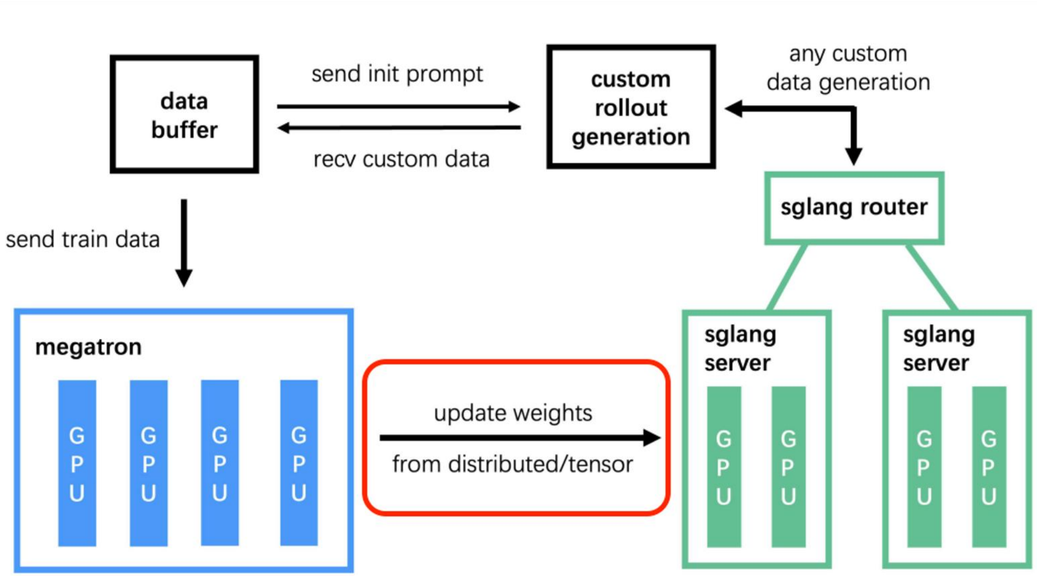
Verl + Megatron-Bridge + SGLang with integrated PEFT support: an open stack for trillion-parameter RL post-training

Single GPU node (8xB200) for RL post-training DeepSeek-v4 1.6T!

Orbit's small overhead in weight synchronization

Weight sync mechanism in RL infra: **copying updated model weights from training side to the inference side**

- Training engine updates weights every optimization step.
- Inference engine generates rollouts, samples actions, but it needs to use the latest policy weights to stay consistent with training.
- These two components often run separately → explicit synchronization is required

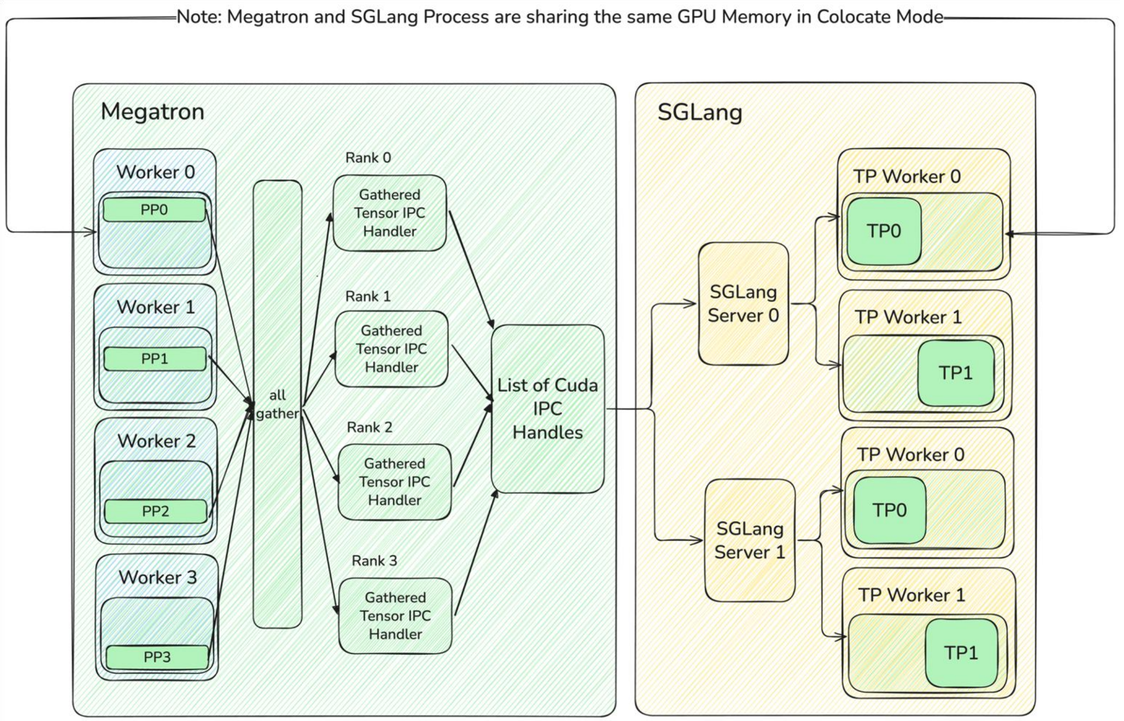


Phase	Duration
Generation (SGLang)	60 seconds
Weight Resharding / Sync	20 seconds (The Waste)
Training (Megatron)	40 seconds

Orbit’s small overhead in weight synchronization

Slime: “collocated” flow

- **Training Ends**: Megatron has the updated weights in the GPU.
- **The "Bridge" Sync (GPU → GPU)**: Instead of going to the CPU, Slime triggers a NCCL Kernel.
- **The Offload**: Once SGLang has the weights it needs in its VRAM, Megatron offloads its Optimizer States (the huge stuff) to the CPU RAM to make room for SGLang’s KV-Cache.
- **In-place resharding**: directly on the GPU
- **Important**: CPU → GPU: only during the optimizer steps, no weight transfer, but huge optimizer state transfer



OFT’s optimizer states are neglectable!

OFT as first-class citizens in RL infrastructure

- **Trivial weight sync:** Only megabyte-scale adapter deltas move between the trainer and the rollout engines
- **Persistent rollout servers.** Because the base model never changes, inference engines stay constantly active across the training lifecycle. The cyclic engine restart between generation and optimization disappears.
- **Viable trillion-parameter scaling.** The memory-heavy optimizer state is bounded to the tiny adapter while the frozen base is in low-precision.
- **Adapter-native async with double-buffered rollout.** Orbit's versioned adapter runtime makes async RL cheap.

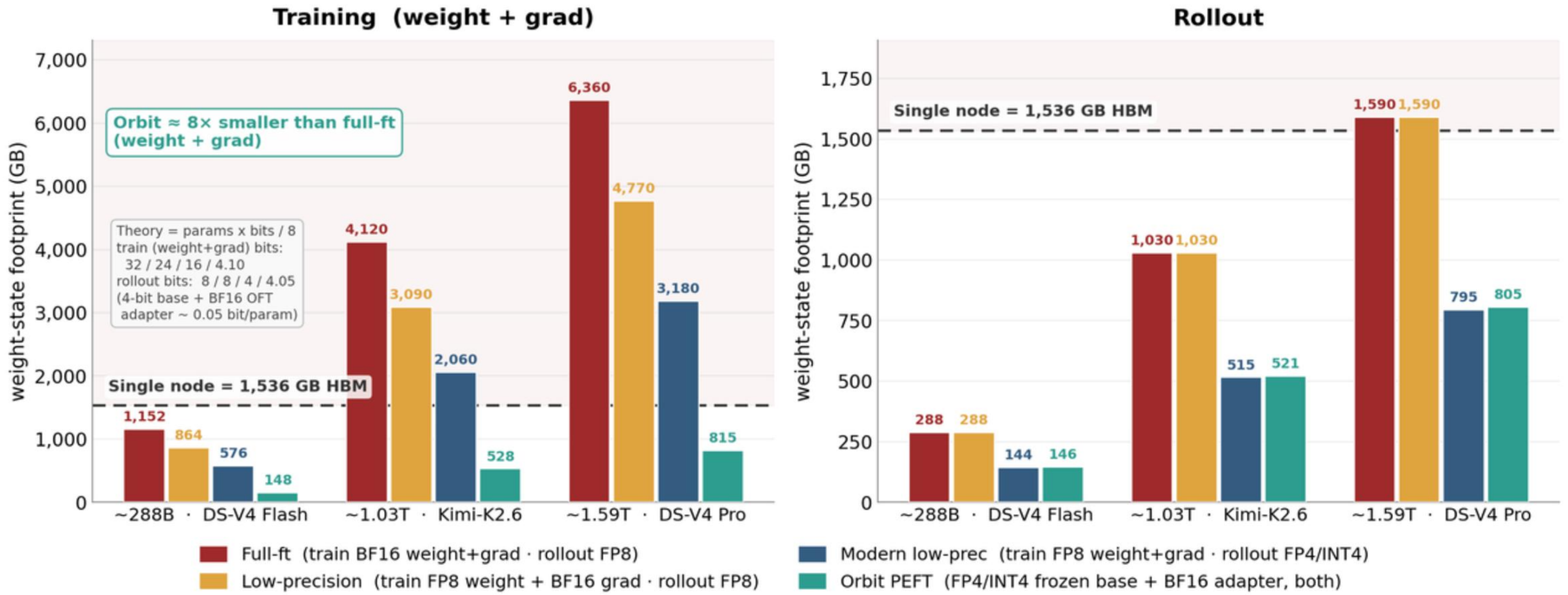
Orbit fully occupies GPU computation



Weight-state memory: why a single node is enough

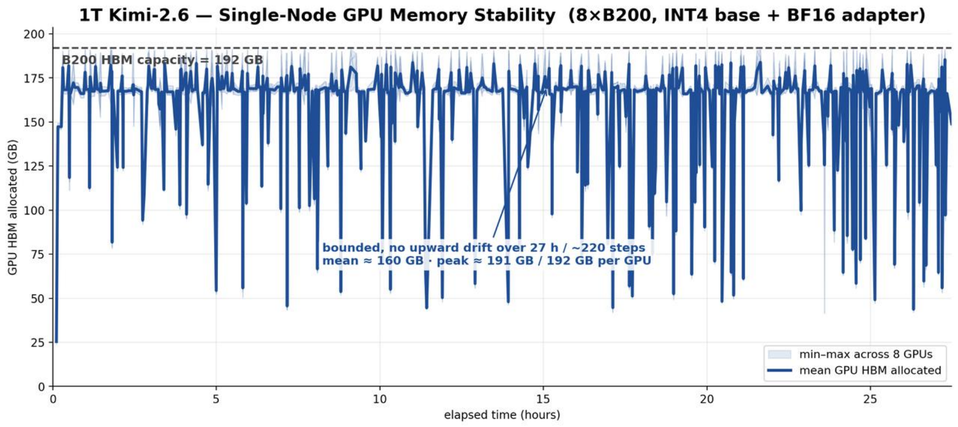
Method	Training	Rollout
Traditional RL full-FT	BF16 forward + BF16 backward	FP8
Low-precision RL full-FT	FP8 forward + BF16 backward	FP8
Modern low-precision RL full-FT (most commonly used)	FP8 forward + FP8 backward	INT4 / FP4
Orbit's PEFT-centric RL	INT4 / FP4 (frozen) + BF16 adapter	same INT4 / FP4 + BF16 adapter

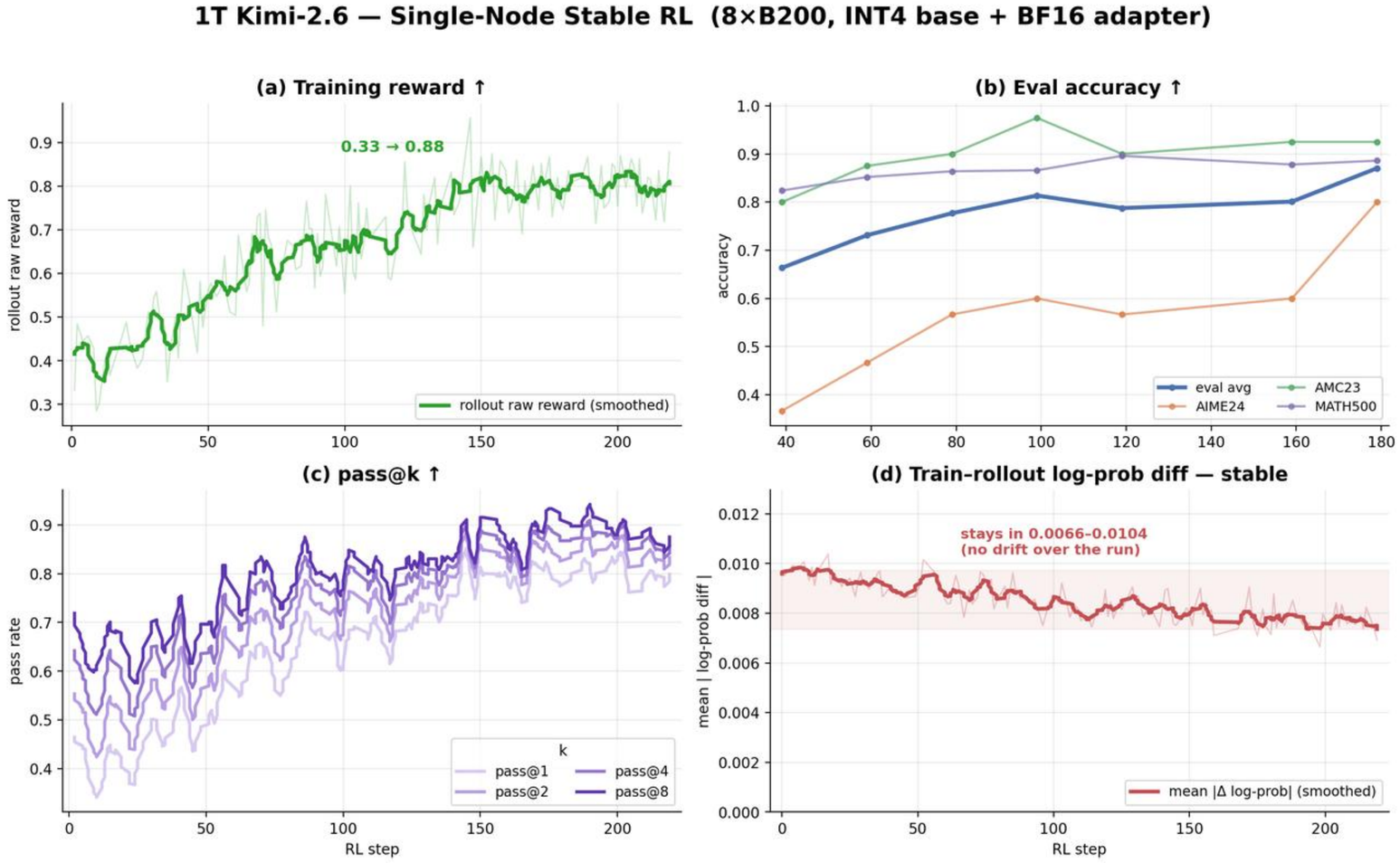
Orbit is the only RL infra that supports single node training of trillion-scale LLMs!



Kimi-K2.6 1T: stable RL on a trillion-parameter model

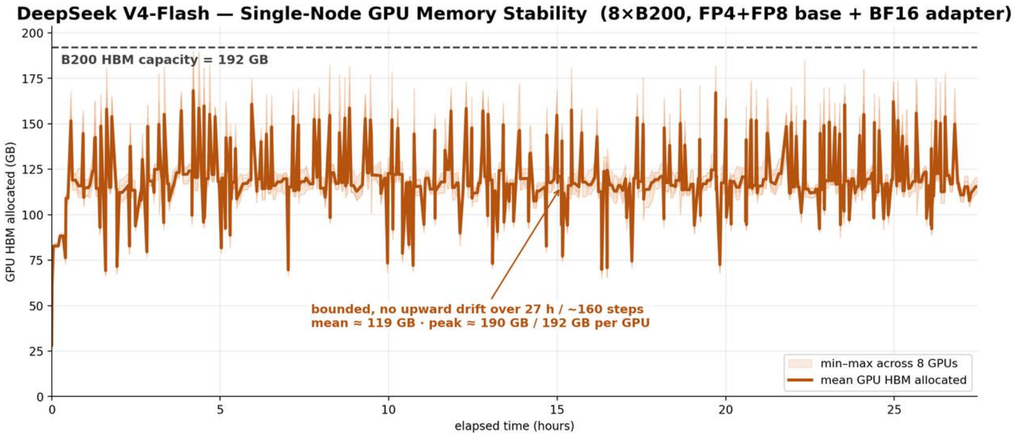
Model	Kimi-K2.6 ~1.03T
Hardware	single 8×B200
Training precision	INT4 base + BF16 adapter
Rollout precision	same INT4 base + BF16 adapter

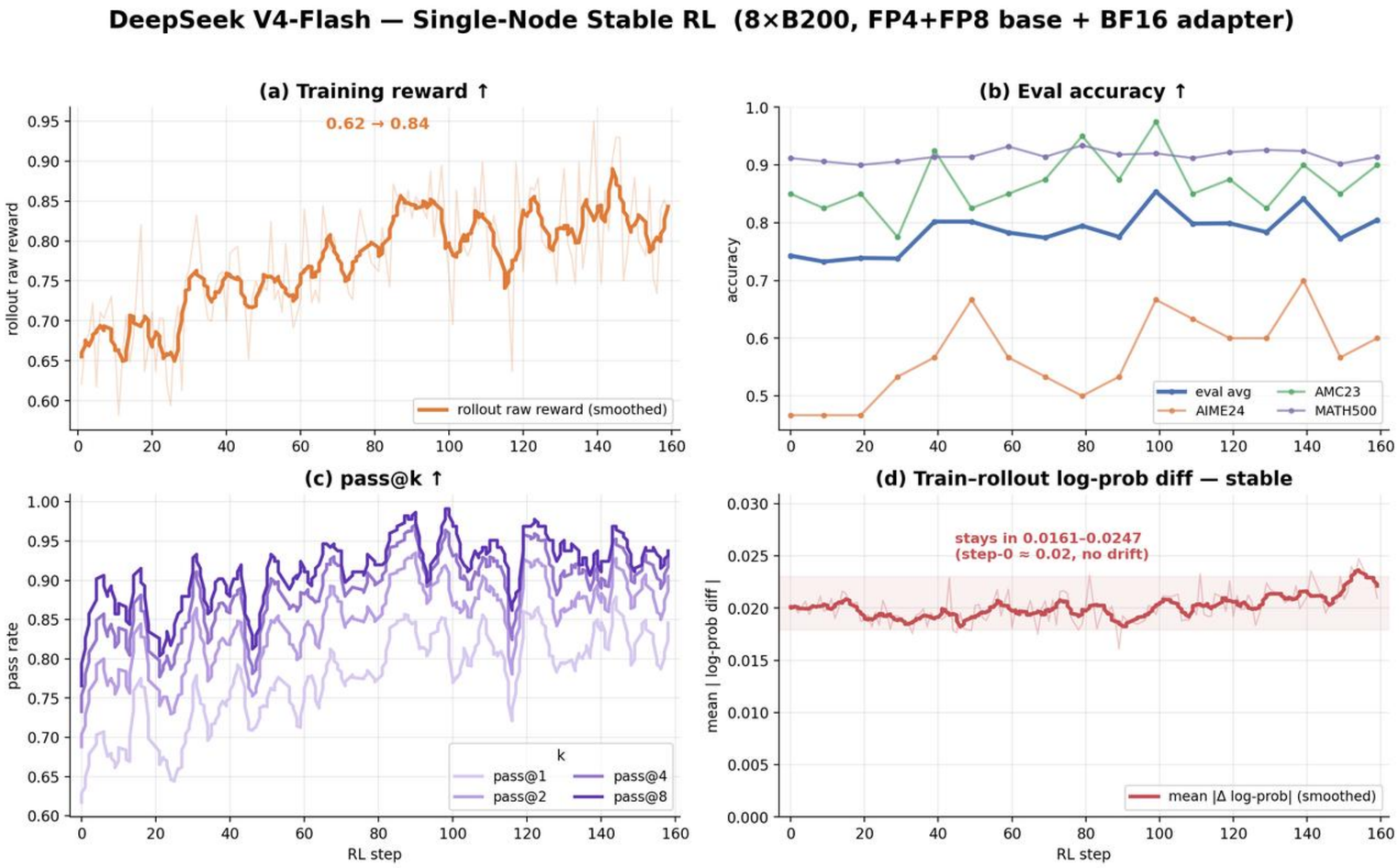




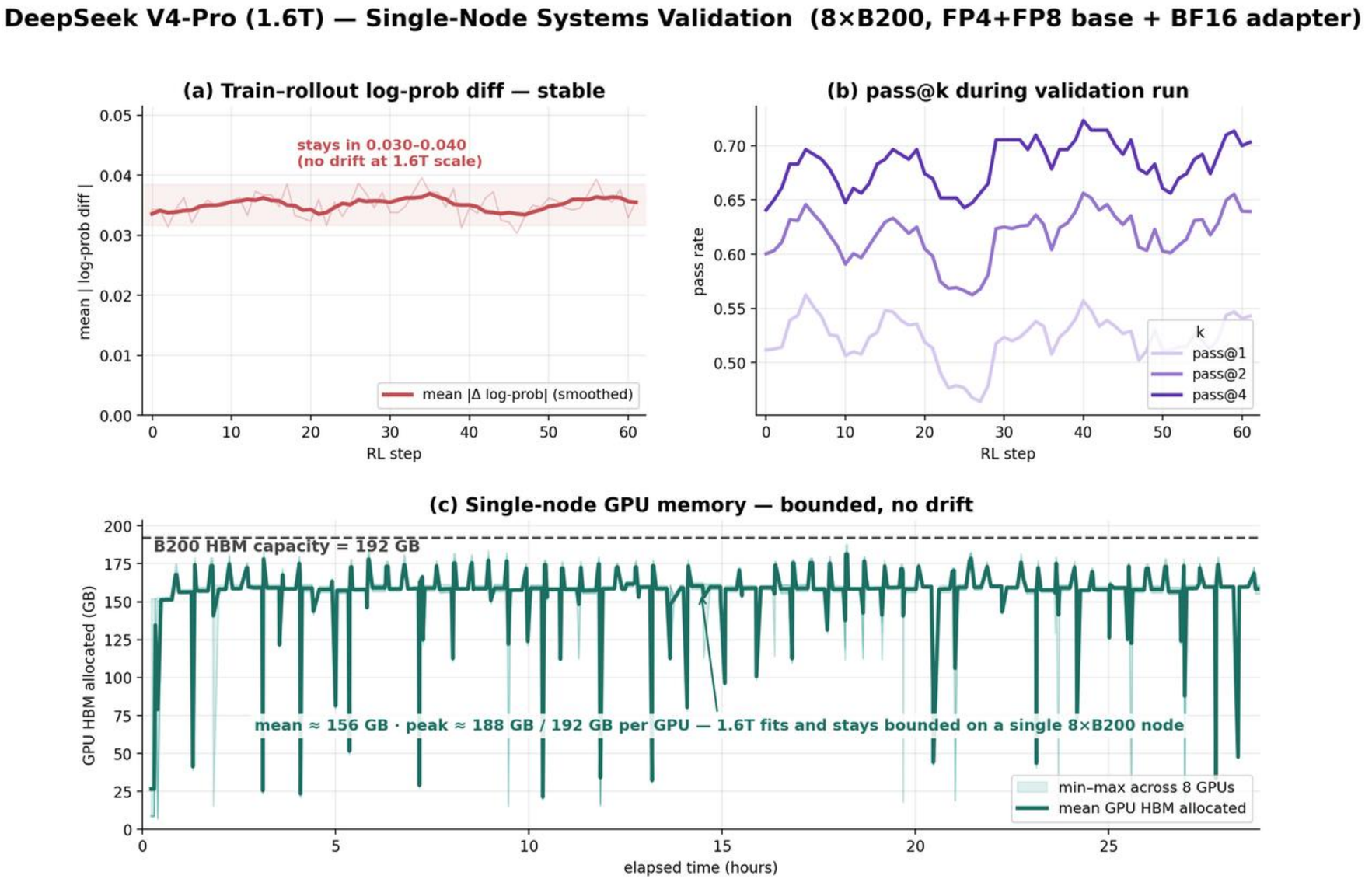
DeepSeek V4-Flash

Model	DeepSeek V4-Flash
Hardware	single 8×B200
Training precision	FP4 base + BF16 adapter
Rollout precision	same FP4 base + BF16 adapter



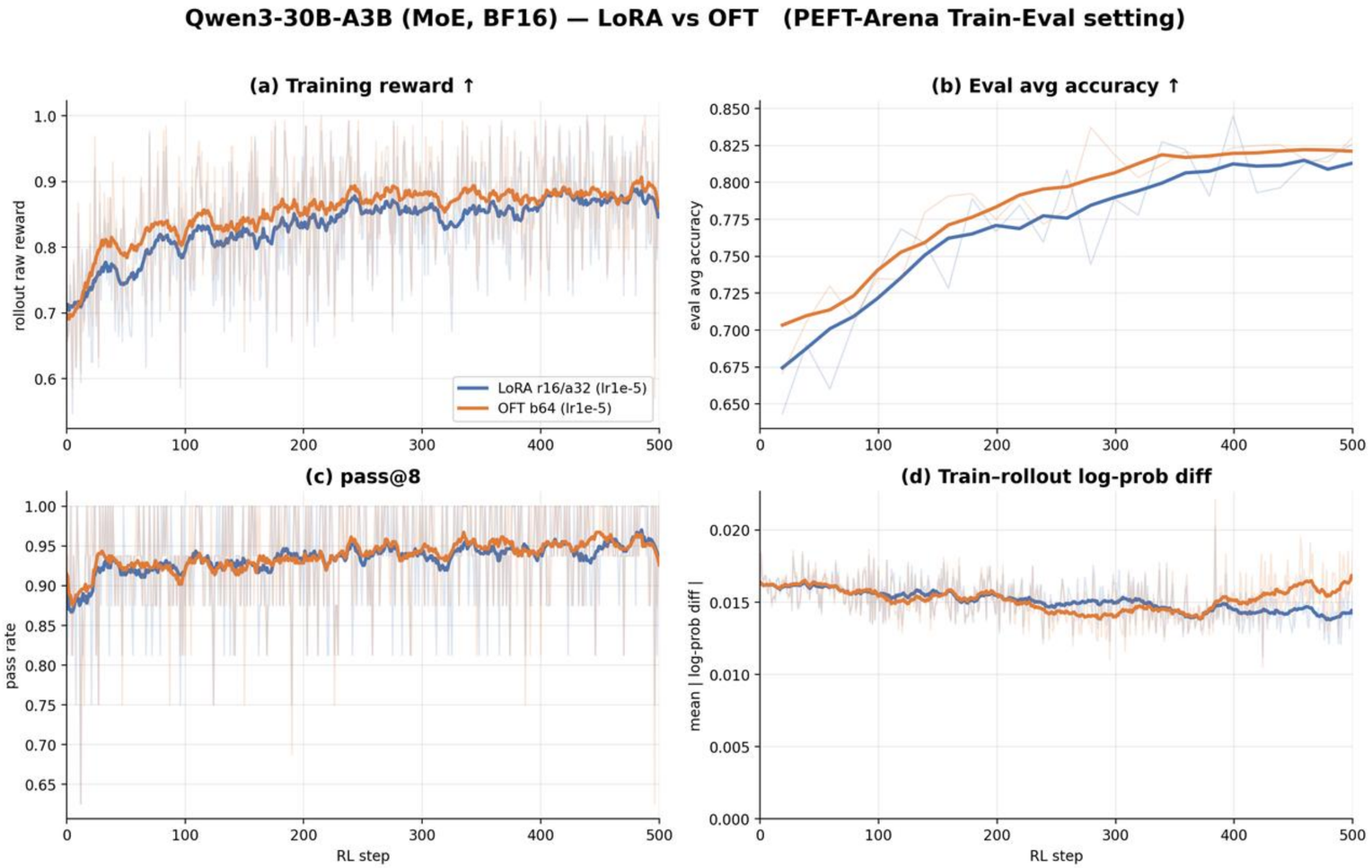


DeepSeek V4-Pro-1.6T



QOFT vs. QLoRA in Orbit

- **Zero additional network communication** overhead when serving OFT.
- **Custom kernels** for split-projection adapters on fused layers.
  - LoRA workaround shares low-rank A, wasting half of the adapter capacity.
  - Custom split-projection kernels that natively apply separate orthogonal rotations



Orbit bridges the train-test gap

Key findings:

- In long-horizon RL training, Orbit consistently improves both the target performance and the general performance!

	Source	Checkpoint	Math Avg pass@1	Math Avg pass@64	General	
OFT merge + VeRL	peft_arena	OFT-b32 step 200	47.90	77.23	47.58	Decrease!
	peft_arena	OFT-b32 step 500	47.83	75.77	46.93	
Orbit	orbit	iter199-merged	45.40	75.70	47.80	Increase!
	orbit	iter499-merged	48.13	80.73	48.40	

Orthogonal Pretraining for a Better Base Model

- To improve the pretraining's flexibility and expressiveness, we use double-sided orthogonal equivalence transformation to pretrain the LLM base model.

PION UPDATE RULE

$$\mathbf{G}_t^{\text{in}} = \mathbf{W}_t^\top \mathbf{G}_t - (\mathbf{W}_t^\top \mathbf{G}_t)^\top, \qquad \mathbf{G}_t^{\text{out}} = \mathbf{G}_t \mathbf{W}_t^\top - (\mathbf{G}_t \mathbf{W}_t^\top)^\top,$$

$$\mathbf{W}_{t+1} = \exp(-\eta \mathbf{G}_t^{\text{out}})\, \mathbf{W}_t\, \exp(-\eta \mathbf{G}_t^{\text{in}}).$$

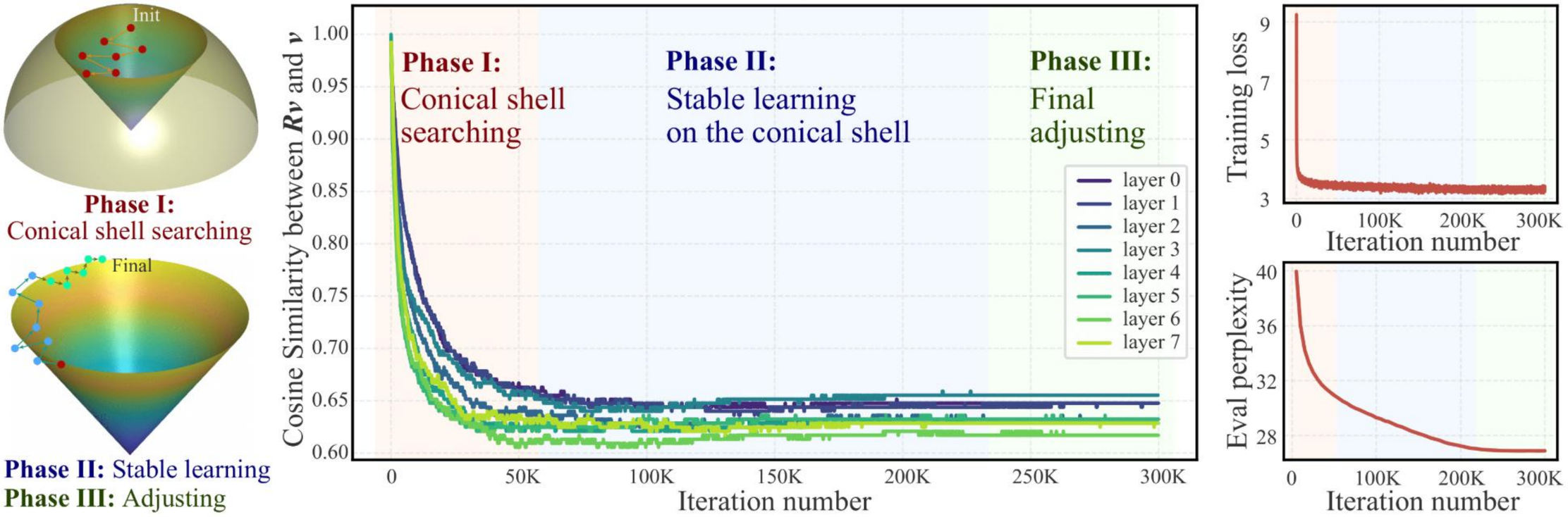


(a) POET's update (b) Pion's update

spherelab.ai/poet
spherelab.ai/poetx
spherelab.ai/pion

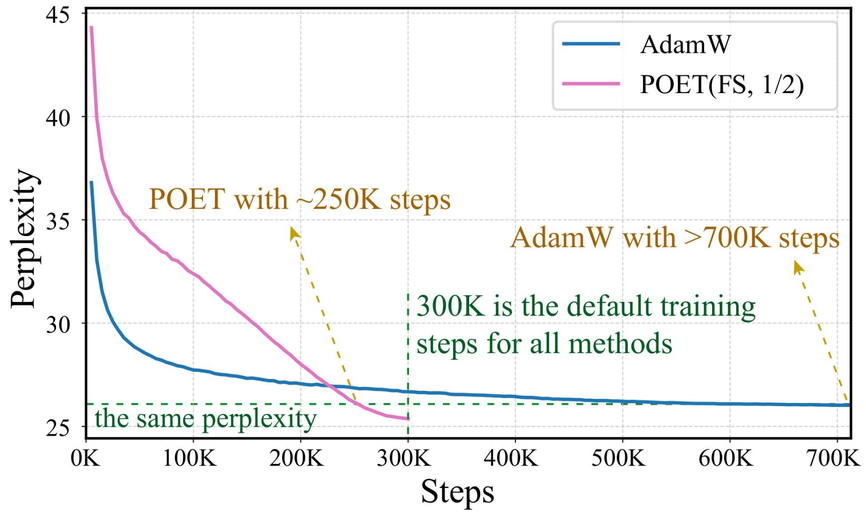
Pion: A Spectrum-Preserving Optimizer via Orthogonal Equivalence Transformation, NeurIPS 2026
POET-X: Memory-efficient LLM Training by Scaling Orthogonal Transformation, ICML 2026
Reparameterized LLM Training via Orthogonal Equivalence Transformation, NeurIPS 2025

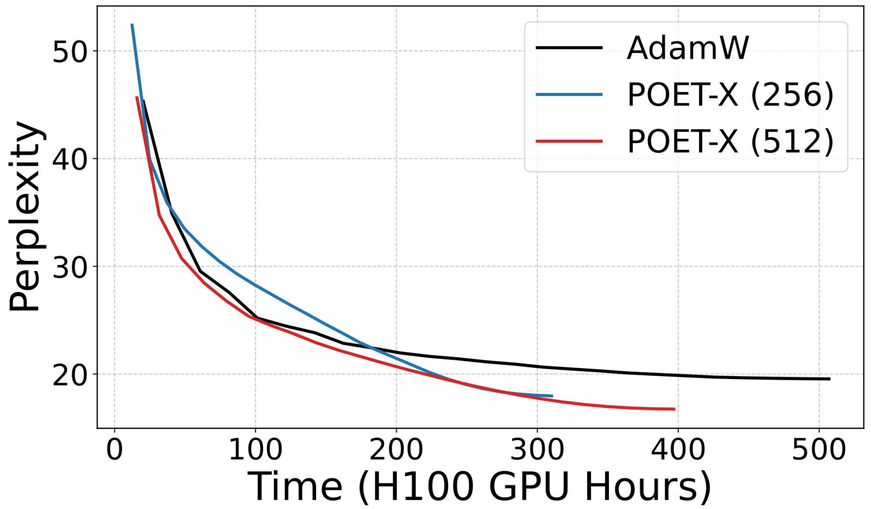
Orthogonal Pretraining

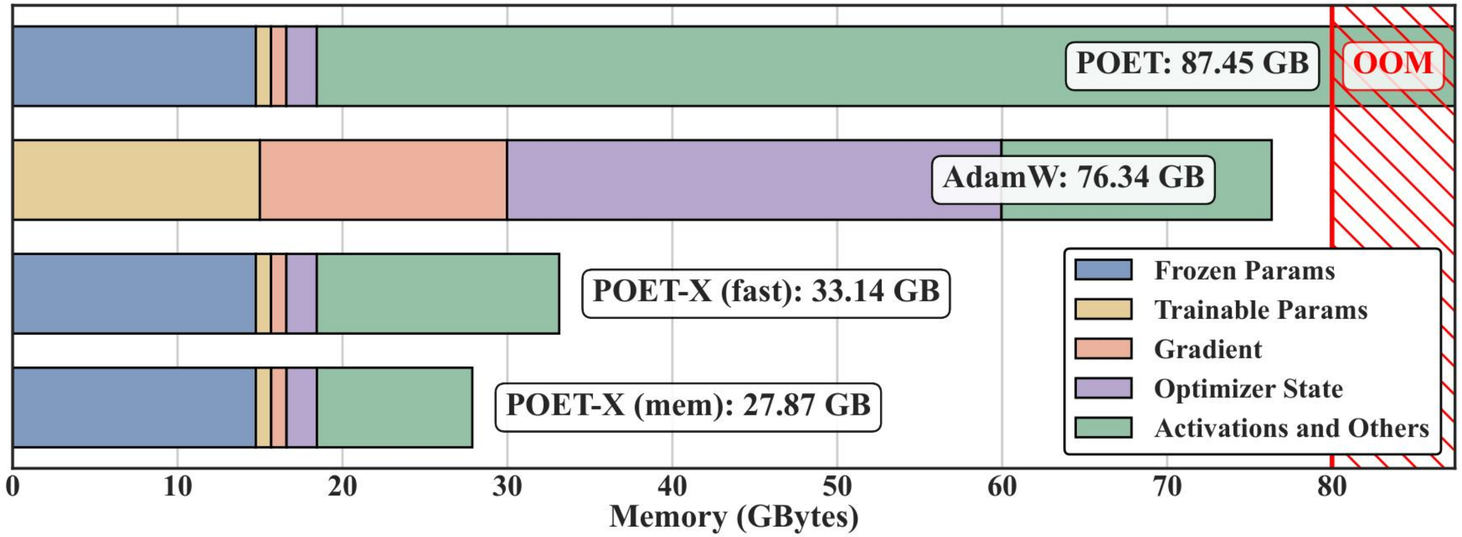


POET's three learning phases. Left: illustration; Middle: angle; Right: loss and validation.

Orthogonal Pretraining

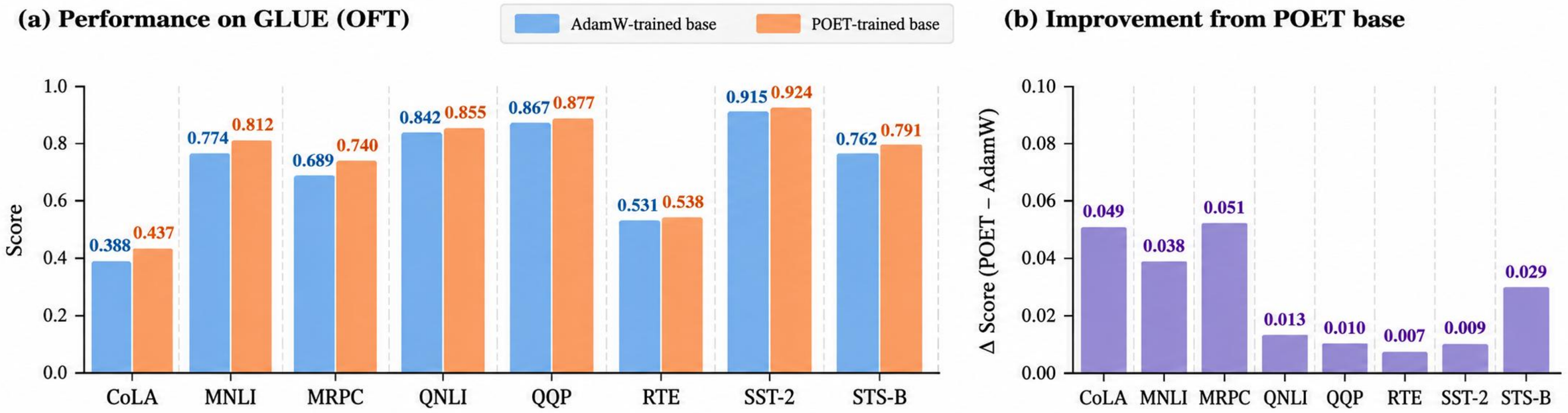






Much more memory-efficient than standard training!

POET-pretrained LLMs are better suited for OFT



Orthogonal pretraining works better with orthogonal post-training!

Summary

- Orthogonal training, as a sparse training pipeline, provides many new opportunities in both algorithm and system level for improving LLM training
- Orthogonal training is inherently suitable for quantized training of LLMs, in both pre-training and post-training.
- Our lab is working on the co-design of training algorithm and system.